

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Разработка мобильного приложения «Рейс-Контроль» для водителей автобусов

Версия 4.1 03 октября 2026 г.

Заказчик: [название транспортной компании]

Разработчик: [ФИО]

2026

Оглавление

1. ОБЩИЕ СВЕДЕНИЯ
 2. ГЛАВНЫЙ ПРИНЦИП РАЗРАБОТКИ
 3. ЧТО ДЕЛАЕТ ПРИЛОЖЕНИЕ
 4. ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ
 5. ОСОБЕННОСТИ РАБОТЫ ПО ВСЕЙ РОССИИ
 6. ГИБКОСТЬ И ГОТОВНОСТЬ К ОБНОВЛЕНИЯМ
 7. ГИБКОСТЬ ХРАНЕНИЯ
 8. ЭТАПЫ РАЗРАБОТКИ
 9. ТЕХНИЧЕСКИЙ СТЕК
 10. СТОИМОСТЬ ИНФРАСТРУКТУРЫ
 11. СТРУКТУРА ХРАНЕНИЯ
 12. ЧТО НЕ ВХОДИТ В ПРОЕКТ
 13. КРИТЕРИИ ПРИЁМКИ
 14. РАСПРОСТРАНЕНИЕ ПРИЛОЖЕНИЯ
 15. ОЖИДАЕМЫЙ РЕЗУЛЬТАТ
 16. СЛЕДУЮЩИЕ ЭТАПЫ
 17. ПЕРЕДАЧА РЕЗУЛЬТАТА
 18. ГАРАНТИИ И ПОДДЕРЖКА
 19. СОГЛАСОВАНИЕ
- ПРИЛОЖЕНИЯ

1. ОБЩИЕ СВЕДЕНИЯ

1.1. Наименование проекта

- Мобильное приложение «Рейс-Контроль» — система фиксации событий рейса водителями автобусов с фотоотчётностью, геолокацией и офлайн-синхронизацией.

1.2. Заказчик

- Транспортная компания, осуществляющая перевозку автомобилей (автобусы) по всей территории Российской Федерации.
- География: любые маршруты — от Калининграда до Владивостока, от Мурманска до Сочи.
- Особенности работы:
- Маршруты по всей территории России.
- Дальние рейсы продолжительностью от нескольких дней до нескольких недель.
- Множественные точки выгрузки в разных городах.

- Часть маршрута проходит вне зон стабильной связи.
- Рейсы могут пересекать несколько часовых поясов.
- Погрузка в одном городе, выгрузка — в одной или нескольких точках.
- Парк: определяется на момент внедрения системы.

1.3. Цель проекта

- Автоматизировать процесс фиксации рейса водителем и убрать ручной сбор фото в мессенджерах.
- Что должно быть достигнуто:
- Водитель фиксирует события рейса прямо в приложении.
- Приложение автоматически записывает координаты и время.
- Водитель фотографирует документы и машины по чек-листу.
- Логист видит все события и фото в реальном времени.
- Фото хранятся на сервере по папкам рейсов.
- Рейс не завершается без выполнения всех условий.
- Работает в любой точке России, включая удалённые регионы.
- Работает без интернета.

2. ГЛАВНЫЙ ПРИНЦИП РАЗРАБОТКИ

Приложение для водителя должно быть проще, чем WhatsApp. Водители работают в перчатках, на солнце, в дороге. Интерфейс должен быть понятен без инструкции.

- Правила:
- Одна главная кнопка на экране.
- Максимум 3 нажатия до сохранения события.
- Крупные кнопки (работа в перчатках).
- Никаких выпадающих списков.
- Никаких технических терминов.
- Русский язык, простые слова.
- Работает без объяснений.
- Работает без интернета.

3. ЧТО ДЕЛАЕТ ПРИЛОЖЕНИЕ

3.1. Сценарий работы водителя

- Водитель открывает приложение.
- Видит свою заявку.
- Нажимает «Начать рейс».
- Едет по маршруту.
- Приехал на точку → нажал «Я на месте» → выбрал тип события → приложение взяло координаты → открылась камера (или выбор: камера / галерея) → сфоткал по чек-листу → сохранил.
- Едет дальше.
- Повторяет для каждой точки.
- Закончил рейс → нажал «Рейс окончен» → ввёл трек-номер → рейс завершён.

Офлайн-режим: если сети нет, событие сохраняется локально. При появлении сети — отправляется автоматически. Водитель видит статус: «В очереди» / «Отправлено» / «Ошибка».

- Кнопка «Отправить всё сейчас» — для принудительной отправки.

3.2. Что видит логист

- В Telegram-боте:
Рейс №123 – Погрузка (Москва)
Водитель: Иванов И.И.
Время: 01.10 14:20

Координаты: 55.7558, 37.6173
<https://maps.google.com/?q=55.7558,37.6173>
Фото: 12

- В веб-кабинете: таблица событий по каждому рейсу + все фото. Рядом с каждым фото — иконка источника (camera / gallery).

4. ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

4.1. Android-приложение водителя (основной клиент)

Функция	Описание
Вход	Телефон + пароль
Список заявок	Только свои
Экран рейса	Большая кнопка «Я НА МЕСТЕ» + 4 кнопки событий
Экран события	Координаты + время + камера + счётчик фото
Геолокация	One-shot GPS при каждом событии + периодический трекинг (15 мин)
Камера	**Основной способ: съёмка через приложение**
Галерея	**Альтернативный способ: загрузка фото из галереи**
Водяной знак	Дата, время, координаты, номер рейса — на **все** фото (камера и галерея)
Офлайн-режим	События копятся локально
Часовые пояса	Время по серверу, отображение в местном
Логирование	Все действия логируются
Обновление	Автопроверка новой версии
Периодический трекинг	ГЛОНАСС-аналог: точки каждые 15 минут во время рейса

Про галерею:

- Галерея **разрешена** как альтернативный способ.
- Водитель может выбрать: сфотографировать через камеру или загрузить из галереи.
- **Причина не запрашивается.**
- В метаданных фиксируется источник: camera / gallery.
- Логист видит иконку источника рядом с фото.
- Водяной знак накладывается на **все** фото.
- Галерея может быть отключена через конфиг (если заказчик решит ограничить).

4.2. iOS-приложение водителя (закладывается, реализуется позже)

- Функционал тот же, что у Android. Отличия:
- Язык: Swift + SwiftUI.
- Локальная БД: Core Data или Realm.
- Камера: AVFoundation.
- Галерея: PhotosUI / PHPicker (альтернативный способ, по аналогии с Android).
- GPS: Core Location (один провайдер).
- Push: APNs.
- Обновление: TestFlight (пилот) / App Store или Enterprise (production).
- Водяной знак накладывается на все фото независимо от источника.
- В метаданных фиксируется source: camera | gallery.
- Возможность отключения галереи через feature flag.
- Требования к API: API не привязан к платформе. Один openapi.yaml для Android и iOS. Заголовок X-Platform (android / ios).

4.3. Веб-кабинет логиста

- Вход.
- Список заявок.
- Создание заявки.
- Карточка рейса со всеми фото.
- Подтверждение закрытия.
- Конфигуратор чек-листов (без релиза приложения).
- Управление релизами (app_releases).
- Просмотр треков (карта).
- Отображение источника фото (camera / gallery) рядом с каждым вложением.

4.4. Telegram-боты

Бот	Кому	Что шлёт
@reiscontrol_bot	Логисту	События с координатами и фото
@reiscontrol_devbot	Разработчику	Логи (успех/ошибка/краш)

Важно: Telegram — внешняя зависимость. Веб-кабинет обязан показывать те же события независимо от Telegram.

5. ОСОБЕННОСТИ РАБОТЫ ПО ВСЕЙ РОССИИ

5.1. Часовые пояса

- Всё хранится в UTC.
- В приложении — местное время.
- В Telegram — время точки события.
- В веб — на выбор.
- Дополнительно (уточнение): время хранится как набор атрибутов:
- device_time_utc — время устройства.
- device_tz_offset_min — смещение зоны.
- elapsed_realtime_ms — монотонные часы (защита от перевода).
- server_time_utc — время сервера.
- received_at_utc — всегда достоверно.
- clock_skew_ms — расхождение.
- time_trust — high / skewed / unknown.
- Порог критичного расхождения: 5 минут.

5.2. Плохая связь

- Событие сохраняется локально.
- Отправка при появлении сети.
- Статус: «В очереди» / «Отправлено».
- Кнопка «Отправить всё сейчас».
- Двухфазная загрузка файлов (presigned URL) — снимает гигабайты с API, позволяет докачку.
- Приоритеты загрузки: сначала превью и метаданные, потом оригиналы.

5.3. Продолжительные рейсы

- Стабильная работа без перезапуска.
- Батарея не разряжается.
- Прогресс не теряется.
- Черновик восстанавливается с диска при сбое.
- Локальные файлы удаляются только после state='accepted' на сервере.

5.4. Разные условия

- Жара, мороз, дождь, снег.
- Работа в перчатках.
- Высокий контраст интерфейса.

5.5. Разные устройства

- Android: minSdk 29 (Android 10+).
- Android-устройства: Xiaomi, Samsung, Huawei, Realme, Honor.
- Работа без Google Play Services (для Huawei) — через порт LocationProvider с реализациями GMS / HMS / Platform.
- iOS: iPhone с iOS 15+ (реализуется позже).

6. ГИБКОСТЬ И ГОТОВНОСТЬ К ОБНОВЛЕНИЯМ

6.1. Общий принцип

Система должна быть спроектирована и реализована так, чтобы любые изменения — от мелких правок до масштабных и грандиозных обновлений — вносились без переписывания системы с нуля и без остановки её работы.

- Это относится ко всем частям системы:
- мобильному приложению водителя (Android, iOS);
- веб-кабинету логиста;
- Telegram-ботам;
- серверной части;
- хранилищу данных;
- форматам обмена данными.

6.2. Требования к обновляемости

- Обновления приложения не должны требовать одновременного обновления всех водителей.
- Старые версии приложения должны продолжать работать с сервером в течение согласованного периода (по умолчанию — 3 месяца).
- Должна быть возможность поэтапного включения новых функций.
- Должна быть возможность отключить новую функцию без выпуска нового релиза (feature flags + kill switch).
- Должна быть возможность добавлять новые типы событий, чек-листов, документов и ракурсов без переписывания системы.
- Должна быть возможность менять состав и порядок шагов чек-листа без выпуска нового релиза приложения.
- Должна быть возможность добавлять новые типы грузов и новые типы заявок.
- Должна быть возможность менять структуру хранения без потери уже сохранённых данных.
- Должна быть возможность добавлять новые каналы уведомлений и новые интеграции.
- Должна быть возможность масштабировать систему при росте числа рейсов, водителей и объёмов фото.

6.3. Требования к обратной совместимости

- Серверная часть должна поддерживать несколько версий мобильного приложения одновременно.
- Изменения в форматах данных не должны ломать уже сохранённые рейсы.
- Должна быть предусмотрена политика минимальной поддерживаемой версии приложения.
- Принудительное обновление допускается только в крайних случаях и должно быть обосновано.
- Android и iOS используют один API. Один `orepari.yaml` для обеих платформ.

6.4. Требования к сопровождаемости

- Код должен быть понятен и сопровождается силами команды, не участвовавшей в первичной разработке.

- Все ключевые решения должны быть задокументированы (ADR).
- Должна быть возможность тестировать изменения до выпуска в продуктив.
- Должна быть возможность откатить неудачное обновление.
- Должны быть предусмотрены среда для проверки изменений и порядок выпуска обновлений.

6.5. Требования к конфигурируемости

- Чек-листы, ракурсы, типы событий, список документов и правила завершения рейса задаются конфигурацией, а не кодом.
- Конфигурация должна позволять добавлять, изменять и отключать элементы без выпуска нового релиза приложения.
- Должна быть возможность задавать разные конфигурации для разных типов рейсов, типов грузов и клиентов.
- Снимок конфигурации фиксируется на рейс (`config_snapshot_id`). Правка чек-листа в середине рейса не меняет уже начатые рейсы.
- Возможность включать/отключать загрузку фото из галереи через конфигурацию (feature flag `gallery_upload_enabled`) без выпуска нового релиза приложения.

7. ГИБКОСТЬ ХРАНЕНИЯ

7.1. Общий принцип

Хранение данных должно быть организовано так, чтобы структура хранения могла расширяться и изменяться без потери уже накопленных данных и без остановки работы системы.

7.2. Требования к хранению

- Структура хранения должна допускать добавление новых типов точек, событий, документов и фото без изменения уже существующих данных.
- Данные должны быть организованы так, чтобы к ним можно было добавлять новые поля и новые сущности без переписывания старых записей.
- Должна быть предусмотрена возможность миграции данных при изменении структуры (`expand/contract`).
- Должна быть возможность хранить данные как структурированно, так и в виде вложений.
- Должна быть возможность изменять правила именования и раскладки файлов без потери связи с уже сохранёнными рейсами.
- Должна быть возможность добавлять новые типы вложений (фото, документы, PDF, подписи, сканы).
- Должна быть возможность хранить несколько версий одного и того же документа или события.
- Должна быть возможность разграничивать доступ к данным по ролям и по рейсам.
- Должна быть возможность выгружать данные по рейсу, по водителю, по периоду и по типу события.
- Должна быть возможность масштабировать хранилище при росте объёмов без изменения логики приложения.
- Для каждого вложения (`attachment`) хранится атрибут `source` (`camera / gallery`) в БД, независимо от физической раскладки файлов.

7.3. Требования к целостности

- Изменение структуры хранения не должно приводить к потере уже сохранённых данных.
- Должна быть возможность восстановить данные после сбоя (PITR, версионирование бакета).
- Должна быть возможность проверить целостность данных по рейсу (`sha256`, реконсильация).
- Должна быть возможность связать событие, фото, документ и рейс между собой независимо от текущей раскладки файлов (связь через ID в БД).

8. ЭТАПЫ РАЗРАБОТКИ

Этап 0. Документы и каркас (3–5 дней)

- ADR-0001..0011.
- Схема БД.
- openapi.yaml v1.
- JSON Schema примитивов.
- «Уточнения к ТЗ».
- Репозиторий GitHub.
- Yandex Cloud: аккаунт, Object Storage (dev / staging / prod), PostgreSQL (dev / staging / prod), YMQ, Lockbox.
- Telegram-боты.
- Домен + SSL.
- Каркас репозитория.
- CI/CD.
- Шаблоны (README, AGENTS, CONVENTIONS, ROADMAP, ARCHITECTURE, .cursorsrules).

Этап 1. Walking skeleton (7–10 дней)

- Авторизация.
- Заявки.
- Экран рейса.
- GPS (one-shot + периодический).
- Камера.
- Галерея (альтернативный источник, feature flag).
- Водяной знак.
- Офлайн-ядро (Room + outbox).
- Синхронизация.
- Telegram-боты.
- Веб-кабинет (одна страница).
- CI/CD с первого дня.
- Что заложить сразу, даже если не используется:
- cargo_units, trip_participants.
- config_snapshots, event_types.
- client_event_id + outbox.
- attachments с sha256, kind, version_of, source, двухфазной загрузкой.
- events как append-only с payload_jsonb + schema_version.
- Заголовки X-App-Version, X-Platform, X-Device-Id, X-Trace-Id.
- app_releases с platform, channel, min_supported.
- location_tracks.
- user_consent.
- Feature flags + очередь уведомлений.

Этап 2. Чек-лист 12 ракурсов (7–10 дней)

- Конфигуратор в веб-кабинете.
- JSON Schema примитивов.
- 12 ракурсов на каждую машину.
- Валидация на клиенте.

Этап 3. Обязательные документы (3–5 дней)

- Список документов из заявки.
- Проверка обязательных.
- Интеграция с чек-листом.

Этап 4. Надёжность (5–7 дней)

- Полевой тест офлайна: перезапуск, обрывы, частичные загрузки, восстановление из черновика.
- Реконсильация.
- Метрики и алерты.

Этап 5. Завершение рейса (3–5 дней)

- Правила завершения.
- Трек-номер.
- Подтверждение логиста.
- Проверка на сервере.

Этап 6. Веб-кабинет (5–7 дней)

- Полноценный кабинет.
- Список заявок.
- Карточка рейса.
- Галерея.
- Конфигуратор.
- Просмотр треков.

Этап 7. Уведомления и логи (3–5 дней)

- Доработка ботов.
- Страница логов.
- Метрики и алерты.

Общий срок: 2–2,5 месяца (пилот). Не как production.

9. ТЕХНИЧЕСКИЙ СТЕК

Компонент	Технология
Android	Kotlin, minSdk 29, Compose
iOS (позже)	Swift, SwiftUI, iOS 15+
Камера Android	CameraX
Камера iOS	AVFoundation
Галерея Android	Photo Picker / MediaStore
Галерея iOS	PhotosUI / PHPicker
GPS Android	LocationProvider (GMS / HMS / Platform)
GPS iOS	Core Location
Локальная БД Android	Room
Локальная БД iOS	Core Data или Realm
Сеть Android	Retrofit + OkHttp
Сеть iOS	URLSession
Бэкенд	Python 3.12 + FastAPI
БД	PostgreSQL managed (вместо YDB)
Хранилище	Yandex Object Storage
Очередь	YMQ или PostgreSQL
Хостинг	Yandex Cloud (Functions / Containers)
Уведомления	Telegram Bot API
Push Android	FCM / HMS
Push iOS	APNs
Веб-кабинет	React + TypeScript + Vite

- Всё в Yandex Cloud. Данные в РФ.

10. СТОИМОСТЬ ИНФРАСТРУКТУРЫ

10.1. На старте (пилот)

Компонент	Цена/мес
Object Storage (dev + staging)	~0 Р (в бесплатных лимитах)
Object Storage (prod)	~100–300 Р
PostgreSQL (dev)	~1 000 Р
PostgreSQL (staging)	~1 500 Р
PostgreSQL (prod)	~2 000–4 000 Р
YMQ	~100 Р
Lockbox	~50 Р
Домен	~500 Р/год
SSL	0 Р
Итого	**~5 000–7 000 Р/мес**

- Для пилота можно начать только с dev + staging → ~2 500 Р/мес.

10.2. При росте (30+ рейсов/мес)

Вариант	Цена/мес
Serverless (PostgreSQL + Functions + API Gateway + Storage)	3 000–6 000 Р
Managed (PostgreSQL + Containers + Storage)	6 000–8 000 Р

10.3. Другие расходы

- Домен + SSL: ~500 Р/год.
- Тестовые устройства: 0 Р (в пилоте используются личные устройства водителей).
- Трафик водителей: компенсируется компанией.

Важно (расчёт объёмов):

- 12 ракурсов × 8 машин + ~10 документов ≈ 110–130 фото на рейс.
- При сжатии до 1920 px / q80 → ~0,5 МБ на фото + превью ~30 КБ → 65–80 МБ на рейс.
- 30 рейсов/мес → ~2 ГБ/мес.
- Бесплатный 1 ГБ Object Storage кончится на 12–15 рейсах.
- Нужны: серверное сжатие, генерация превью, лимит разрешения на клиенте, lifecycle-политика (холодное хранилище через 90 дней).

11. СТРУКТУРА ХРАНЕНИЯ

- Физическая раскладка в Object Storage — иммутабельная, по attachment_id:

```
s3://reiscontrol-prod/
  raw/{trip_id}/{event_id}/{attachment_id}/original.jpg
  derived/{attachment_id}/w1920.jpg
  derived/{attachment_id}/thumb.jpg
  docs/{trip_id}/{attachment_id}/scan.pdf
  exports/{export_job_id}/trip-{trip_id}.zip
  manifests/{trip_id}/meta.json
  apk/reiscontrol-{versionName}-{versionCode}.apk
  ipa/reiscontrol-{versionName}-{versionCode}.ipa
```
- Красивая раскладка из ТЗ (/orders/123/loading_moscow/...) реализуется как виртуальная структура экспорта: сервер по ID собирает ZIP с нужными папками. Правила раскладки — конфигурация (export_layout).

- manifests/{trip_id}/meta.json — содержит для каждого attachment поле source (camera / gallery).
- Правила:
- Ключ неизменяем (immutable).
- Связь «рейс ↔ событие ↔ фото ↔ документ» живёт в БД по ID.
- Lifecycle: raw/ → холодное хранилище через 90–180 дней.
- Версионирование бакета включено.
- sha256 в БД позволяет проверить целостность.

12. ЧТО НЕ ВХОДИТ В ПРОЕКТ

Что	Почему
Постоянный GPS-трекинг (24/7)	Не нужен. Периодический (15 мин) во время рейса — да
Карта маршрута	Не нужна на старте
Конструктор типов грузов	Один тип — автовоз
Интеграция с 1С	Этап 2
ЭТрН	Этап 2
iOS-релиз	Закладывается, реализуется позже
Сложная аналитика	Этап 3

13. КРИТЕРИИ ПРИЁМКИ

13.1. Из ТЗ (базовые)

Этап	Критерий
1	Водитель проходит цепочку «Я на месте» → GPS → камера → фото → отправка
1	Логист получает событие с координатами и фото в Telegram
1	Разработчик получает логи в Telegram
1	Фото хранится в папке рейса
1	Водяной знак виден на фото
1	Водитель может загрузить фото из галереи; водяной знак накладывается
1	Логист видит иконку источника (`camera` / `gallery`) рядом с фото
2	Для каждой машины выполнены 12 ракурсов
2	До завершения фотосъёмки событие не сохраняется
3	До съёмки всех обязательных документов событие не сохраняется
4	В авиарежиме событие сохраняется локально
4	После появления сети отправляется автоматически
5	Рейс нельзя завершить без фото
5	Рейс нельзя завершить без трек-номера
6	Логист создаёт заявку
6	Водитель видит заявку
6	Логист видит все события с фото
7	Все события приходят в Telegram
7	Логи разработчика приходят в отдельный бот

13.2. Дополнительные (критичные)

- Обновить приложение с незагруженными фото → проверить сохранность.
- Повторить отправку операции → получить одно событие (идемпотентность).
- Изменить чек-лист во время офлайн-рейса → сохранить прежние требования рейса.
- Восстановить БД и файлы из резервной копии → открыть полный рейс.
- Периодический трекинг → точки записываются каждые 15 минут во время рейса.
- Согласие на ПДн → при первом входе, отдельно на геолокацию и трекинг.
- Отключить галерею через конфиг во время активного рейса → водитель видит только камеру, ранее загруженные из галереи фото сохраняются.
- Загрузить фото из галереи без причины → метаданные содержат source: gallery, причина не запрашивается.
- При отключённом через конфиг галерее — возможность загрузки из галереи отсутствует в UI.

13.3. Из требований гибкости

- Изменение чек-листа, ракурсов или списка документов не требует релиза приложения.
- Добавление нового типа события не требует переписывания системы.
- Изменение структуры хранения не приводит к потере сохранённых данных.
- Старая версия приложения продолжает работать после обновления сервера.

14. РАСПРОСТРАНЕНИЕ ПРИЛОЖЕНИЯ

14.1. Android

- На первом этапе:
- APK-файл напрямую водителю.
- Установка вручную.
- Обновление — новый APK через API (app_releases).
- В дальнейшем:
- Публикация в RuStore.
- Обновление без потери данных и без остановки работы.
- Каналы обновления:
- direct — APK напрямую (Object Storage).
- rustore — RuStore.
- При обновлении приложения настройка «галерея разрешена/запрещена» сохраняется (приходит с сервера через конфиг).

14.2. iOS (закладывается, реализуется позже)

- Способы распространения:
- TestFlight — для пилота (бесплатно, до 10 000 тестеров).
- Apple Developer Enterprise Program — для сотрудников компании (\$299/год).
- App Store — для публикации (модерация Apple).
- Каналы обновления:
- testflight — TestFlight.
- appstore — App Store.
- enterprise — Enterprise Program.

15. ОЖИДАЕМЫЙ РЕЗУЛЬТАТ

15.1. Для компании

- Убраны мессенджеры из операционного контура.
- Все фото структурированы по рейсам.
- Логист видит события в реальном времени.

- Есть доказательная база.
- Снижено время на закрытие рейсов.
- Система готова к развитию и масштабным обновлениям.
- Есть периодический трекинг (ГЛОНАСС-аналог) для контроля рейсов.

15.2. Для водителя

- Понятное приложение.
- Не надо искать, куда отправить фото.
- Работает без интернета.
- Все данные сохраняются автоматически.

15.3. Для логиста

- Все уведомления в Telegram.
- Все фото в одном месте.
- Быстрый поиск.
- Подтверждение закрытия в один клик.
- Просмотр треков на карте.

16. СЛЕДУЮЩИЕ ЭТАПЫ

- Интеграция с оператором ЭПД (ЭТрН).
- Сканер документов (ML Kit).
- Акт осмотра ТС с PDF.
- Расширенный чек-лист (24 ракурса).
- Второй тип груза.
- Интеграция с 1С.
- Публикация в RuStore.
- Реализация iOS-клиента (TestFlight / App Store / Enterprise).
- Все перечисленные этапы должны быть реализуемы без переписывания системы с нуля и без потери уже накопленных данных.

17. ПЕРЕДАЧА РЕЗУЛЬТАТА

- Что передаётся заказчику:
- Исходный код (GitHub-репозиторий).
- Доступ к Yandex Cloud (аккаунт заказчика).
- Доступ к Telegram-ботам.
- Документация (ТЗ, архитектура, ADR, схема БД, openapi.yaml, JSON Schema).
- Инструкции (для водителя, логиста, администратора).
- Права:
- Исключительные права на код — заказчику.
- Аккаунты облака, домена, ботов — на заказчика.

18. ГАРАНТИИ И ПОДДЕРЖКА

- Гарантийная поддержка: 3 месяца.
- Критические баги — 24 часа.
- Поддержка после гарантии: по отдельному соглашению.
- Ориентир: 25 000–40 000 ₽/мес.

19. СОГЛАСОВАНИЕ

Реквизит	Заказчик	Исполнитель
Должность		

ФИО		
Подпись		
Дата	«__» _____ 2026 г.	«__» _____ 2026 г.

- М.П.

ПРИЛОЖЕНИЯ

- Приложение А. Макеты экранов.
- Приложение Б. Схема БД (docs/04-db/schema.md).
- Приложение В. API-документация (docs/05-api/openapi.yaml).
- Приложение Г. Структура папок.
- Приложение Д. Список поддерживаемых устройств.
- Приложение Е. ADR-0001..ADR-0011.
- Приложение Ж. JSON Schema примитивов.
- Приложение З. Архитектура системы v1.0.

Журнал изменений

Версия	Дата	Изменения
4.0	02.10.2026	Базовая версия
4.1	03.10.2026	Галерея разрешена как альтернативный способ загрузки фото. Водяной знак накладывается на все фото (камера и галерея). В метаданных фиксируется источник ('camera' / 'gallery'). Логист видит иконку источника. Причина загрузки из галереи не запрашивается. Возможность отключения галереи через конфиг (feature flag 'gallery_upload_enabled'). Обновлены разделы 3.1, 3.2, 4.1, 4.2, 4.3, 6.5, 7.2, 8, 9, 11, 13.1, 13.2, 14.1.